

# Smart Teams API Reference

Every endpoint exposed by the platform: what it does, how to call it, and what happens server-side when you do.

**Base URL:** your deployed backend origin, e.g. `https://smartteams-admin.onrender.com`

**Versioned path (recommended for integrations):** prefix every path below with `/api/v1` instead of `/api`

**Authentication — two ways:**

- Human login: Authorization: Bearer <JWT> obtained from POST `/auth/login` (24h expiry, single active session per account)
- Machine-to-machine: Authorization: Bearer <stk\_live\_...> obtained from POST `/tenant/service-accounts` — a long-lived key scoped to an explicit privilege list, for partner integrations that should not hold a human login

**Rate limits:** 300 requests/minute per authenticated caller (or per IP before authentication); 10 requests per 15 minutes on `/auth/login` specifically. A limit hit returns 429 with `Retry-After` and `RateLimit-*` headers.

**Versioning policy:** `/v1` is stable — existing fields/endpoints never removed or changed in place. Additive changes may ship to `/v1` any time. Any breaking change ships as `/v2` first, with `/v1` kept running in parallel.

# Authentication

Login, password reset, Google Sign-In, and session logout. These are the only endpoints a caller can reach with no bearer token at all (aside from Health and Tenancy Onboarding).

POST  
T

/api/auth/login

PUBLIC

**What it does:** Logs a user in with email + password.

**How it is called:** POST JSON body { email, password, deviceId } (deviceId required for roles that clock in — it pins the account to one device).

**What happens when called:** Verifies the password hash, checks the tenant is not suspended, issues a new session (invalidating any previous session for that user — single-session-per-account by design), and returns a 24-hour JWT plus the user's profile. If the account has a temporary password, returns a short-lived tempToken and requirePasswordChange:true instead.

POST  
T

/api/auth/google

PUBLIC

**What it does:** Logs in with a Google ID token, for an account that was already created another way.

**How it is called:** POST JSON body { credential, deviceId } where credential is a Google Identity Services ID token.

**What happens when called:** Verifies the Google token server-side, matches it to an existing account by email, and issues a session exactly like /auth/login. Does not create new accounts.

POST  
T

/api/auth/reset-password

BEARER JWT (TEMP TOKEN)

**What it does:** Sets a permanent password, completing the forced first-login password change flow.

**How it is called:** POST with Authorization: Bearer <tempToken> (the short-lived token returned by /auth/login when a temp password was used) and JSON body { newPassword }.

**What happens when called:** Hashes and stores the new password, clears the mustChangePassword flag, and issues a normal full session token.

POST  
T

/api/auth/forgot-password

PUBLIC

**What it does:** Requests a self-service password reset email.

**How it is called:** POST JSON body { email }.

**What happens when called:** If the email matches an account, emails a reset link with a signed token. Always returns the same generic success message whether or not the email matched, to avoid leaking which emails have accounts.

POST

/api/auth/forgot-password/confirm

PUBLIC

**What it does:** Completes a self-service password reset using the link from the email above.

**How it is called:** POST JSON body { token, newPassword }.

**What happens when called:** Verifies the reset token, sets the new password hash, and invalidates the token so it cannot be reused.

POST

/api/auth/logout

BEARER JWT

**What it does:** Ends the caller's own current session.

**How it is called:** POST with Authorization header, no body needed.

**What happens when called:** Clears the session ID stored on the user's row, so that JWT (and any other JWT that shared the same session) is rejected on the next request even though it has not technically expired yet.

## Health & Documentation

Liveness/readiness probes for uptime monitors and orchestrators, plus this API's own machine-readable spec. All public by design.

GET

/api/health

PUBLIC

**What it does:** Fast liveness check — is the process up at all.

**How it is called:** Plain GET, no auth, no parameters.

**What happens when called:** Returns {status:'ok'} immediately. Touches no database or external service — used by Render/Fly's own health-check polling to decide whether to keep routing traffic to this instance.

GET

/api/health/db

PUBLIC

**What it does:** Readiness check that the database is actually reachable.

**How it is called:** Plain GET, no auth.

**What happens when called:** Runs SELECT 1 against Postgres. Returns 200 if it succeeds, 503 if the database is unreachable — also used as a keep-alive ping so a free-tier managed database (e.g. Neon) does not suspend from inactivity.

**GET**    **/api/health/face**

**PUBLIC**

**What it does:** Readiness check for the separate Python face-recognition microservice.

**How it is called:** Plain GET, no auth.

**What happens when called:** Calls the face-service's own /health endpoint and reports whether it is reachable and whether its ML model has finished loading. The frontend uses this to decide whether to even attempt a camera-based KYC/check-in flow.

**GET**    **/api/openapi.json**

**PUBLIC**

**What it does:** The raw OpenAPI 3.0 specification for this entire API, machine-readable.

**How it is called:** Plain GET, no auth.

**What happens when called:** Returns the spec object (see apps/admin/openapi.ts) as JSON — this is documentation, not a second source of truth; it mirrors the actual routes.

**GET**    **/api/docs**

**PUBLIC**

**What it does:** Interactive Swagger UI built from the spec above.

**How it is called:** Open in a browser.

**What happens when called:** Renders a clickable, human-browsable version of /api/openapi.json — this is the primary reference for a partner integrating against this API.

## Tenancy Onboarding

The public, no-account-needed sign-up flow for a brand new company, plus the super-admin approval step that provisions it.

**POST**  
**T**    **/api/tenancy/request**

**PUBLIC**

**What it does:** A prospective customer submits a request to sign their company up.

**How it is called:** POST JSON body { companyName, email, numEmployees, plan }.

**What happens when called:** Stores the request and notifies the super admin(s). Does not create a tenant or any user account yet — that only happens once a super admin approves it (see /api/super/approve below).

## Attendance & Check-in

Office check-in/out, the three-step biometric verification flow, corrections, and continuous presence heartbeats. The authoritative record-of-truth endpoint is the plain POST /api/attendance.

**GET**     **/api/attendance/today**

**BEARER JWT**

**What it does:** The caller's own attendance state for today.

**How it is called:** Plain GET.

**What happens when called:** Returns whether they have checked in yet, at what time, and whether that check-in is still pending manager approval (e.g. a late arrival).

**GET**     **/api/attendance/percentage**

**BEARER JWT**

**What it does:** The caller's monthly attendance percentage against the tenant's configured minimum.

**How it is called:** Plain GET.

**What happens when called:** Computes days present vs. working days so far this month and compares it to the tenant's minAttendancePercent policy.

**GET**     **/api/attendance/mine**

**BEARER JWT**

**What it does:** The caller's own attendance history.

**How it is called:** GET, optional ?limit= query parameter.

**What happens when called:** Returns a capped list of the caller's own past attendance log rows, newest first.

**GET**     **/api/attendance/challenge**

**BEARER JWT**

**What it does:** Issues a fresh, random liveness challenge before the camera opens.

**How it is called:** Plain GET.

**What happens when called:** Server picks 3 random actions (e.g. blink, turn\_left, smile) from the pool this user actually verified during their own KYC enrollment, stores them server-side against the user's ID with a short expiry, and returns them to display on screen. The later verify-face call is checked against exactly this stored set, not anything the client claims to have shown.

**POST**  
**T**     **/api/attendance/verify-face**

**BEARER JWT**

**What it does:** Step 1 of the check-in flow — submits a camera burst for identity + liveness + challenge-response verification.

**How it is called:** POST JSON body { images: [burst of JPEG data URLs], mode? }.

**What happens when called:** Forwards the images to the face-recognition microservice, which returns a face embedding, a liveness score, and which requested actions it actually detected. This route then compares the embedding against the user's own enrolled embeddings (cosine similarity, threshold currently 0.5), checks liveness  $\geq 0.6$ , and checks a majority of the requested challenge actions were confirmed. On success, mints a short-lived signed token (a few minutes) asserting "this specific check just happened" — required by the final /api/attendance submit below. On failure, returns a fallback list of guided actions to retry.

**POST**  
**T**

**/api/attendance/verify-location**

**BEARER JWT**

**What it does:** Step 2 (optional preview) — a fast-fail check of whether the caller is inside the office geofence.

**How it is called:** POST JSON body { lat, lng, token } where token is the face-pass token from step 1.

**What happens when called:** Compares the given coordinates against the tenant's configured office location/radius and returns pass/fail plus the distance. Not authoritative — the final submit re-checks this itself regardless.

**POST**  
**T**

**/api/attendance/verify-network**

**BEARER JWT**

**What it does:** Step 3 (optional preview) — a fast-fail check of whether the caller is on the office Wi-Fi/IP (only relevant if the tenant enabled that policy).

**How it is called:** POST JSON body { simulatedIp?, token }.

**What happens when called:** Compares the caller's resolved IP against the tenant's configured office IP. Also just a preview; not authoritative.

**POST**  
**T**

**/api/attendance**

**BEARER JWT**

**What it does:** THE authoritative check-in/check-out submit — the only endpoint that actually records attendance.

**How it is called:** POST JSON body { token, deviceId, lat, lng, clientTimestamp?, explanation?, mode?: 'office'|'wfh', wfhReason? } where token is the face-pass token from verify-face.

**What happens when called:** Re-validates everything server-side from scratch regardless of what the earlier preview steps said — face match, liveness, device pinning, clock-drift, geofence (or WFH home-location distance if mode is 'wfh'), and Wi-Fi if enabled. Writes an attendance\_logs row (status approved/pending/rejected), fires a webhook event to any subscribed integration, and if the check-in was late or is a WFH request requiring approval, notifies whoever holds the approval privilege.

**POST**  
**T**

**/api/attendance/checkout**

**BEARER JWT**

**What it does:** A quick end-of-day checkout with no re-scan required.

**How it is called:** POST JSON body { clientTimestamp? }.

**What happens when called:** Writes a check\_out attendance\_logs row. Blocked if the caller currently has an active break running — they must end the break first.

**POST**  
**T**

**/api/attendance/heartbeat**

**BEARER JWT**

**What it does:** A continuous presence ping sent periodically while checked in.

**How it is called:** POST JSON body { lat?, lng?, simulatedIp?, deviceId? }.

**What happens when called:** Updates the user's last-known location/timestamp and flags a warning (not a hard rejection) if it detects geofence or Wi-Fi drift mid-shift.

POST  
T

/api/attendance/corrections

BEARER JWT

**What it does:** Requests a correction for a missed or incorrect attendance record.

**How it is called:** POST JSON body { requestType, requestedDate, requestedTime?, reason }.

**What happens when called:** Creates a correction request row, visible to the approver queue below, and emails whoever holds the approval privilege.

GET

/api/attendance/corrections/mine

BEARER JWT

**What it does:** The caller's own correction request history.

**How it is called:** Plain GET.

**What happens when called:** Returns their own past correction requests and their current status.

## Biometric KYC (Enrollment)

One-time guided-pose biometric enrollment that every attendance verification is later checked against. All detection happens server-side, in the separate face-recognition microservice.

POST  
T

/api/kyc

BEARER JWT

**What it does:** Submits the full guided-pose enrollment burst, completing KYC.

**How it is called:** POST JSON body { actions: { [poseName]: string[] of JPEG data URLs }, deviceId } covering all 8 guided poses (look\_center, turn\_left, turn\_right, look\_up, look\_down, smile, open\_mouth, blink).

**What happens when called:** Sends every frame to the face-recognition service, which returns one embedding per frame that actually contained a detected face plus a per-pose verified/failed log. Stores every accepted embedding on the user's account (used as the comparison set for every future check-in), pins the device ID, and marks isKycCompleted. If some poses could not be confirmed, returns 422 with just the failed ones so the client can retry only those.

POST  
T

/api/kyc/verify-step

BEARER JWT

**What it does:** Verifies a single retried pose (used to redo just the poses that failed above, not the whole burst).

**How it is called:** POST JSON body { action, images }.

**What happens when called:** Runs the same per-pose detection as above for one action only, appends any newly-accepted embeddings, and updates that one entry in the pose action log.

## Work From Home (WFH)

An alternate attendance mode: instead of the office geofence, distance is checked against a registered home location, with its own eligibility policy and change-request approval flow.

**GET**     **/api/attendance/wfh/eligibility**

**BEARER JWT**

**What it does:** Checks whether the caller can start a WFH check-in today, before the camera even opens.

**How it is called:** Plain GET.

**What happens when called:** Evaluates the tenant's WFH policy (is it enabled tenant-wide, is the caller's role allowed, is today an allowed weekday, has the monthly quota been used up) plus whether a home location is registered yet.

**GET**     **/api/attendance/wfh/home-location**

**BEARER JWT**

**What it does:** Returns the caller's currently registered home location, if any.

**How it is called:** Plain GET.

**What happens when called:** Simple lookup, no side effects.

**POST**  
**T**     **/api/attendance/wfh/register-home**

**BEARER JWT**

**What it does:** One-time first registration of a home location.

**How it is called:** POST JSON body { lat, lng, accuracy? }.

**What happens when called:** Stores the location. Fails with 400 if one is already registered — changing it afterward requires the approval flow below, specifically so an employee cannot silently move their "home" to somewhere else without a manager knowing.

**POST**  
**T**     **/api/attendance/wfh/location-change-request**

**BEARER JWT**

**What it does:** Requests a change to an already-registered home location.

**How it is called:** POST JSON body { lat, lng, accuracy?, reason? }.

**What happens when called:** Creates a pending request visible to the approver queue below and notifies whoever holds the approval privilege; the old location stays active until approved.

**GET**     **/api/attendance/wfh/location-change-requests/mine**

**BEARER JWT**

**What it does:** The caller's own location-change request history.

**How it is called:** Plain GET.

**What happens when called:** Read-only lookup.

**GET**    **/api/tenant/wfh/location-change-requests**

**BEARER JWT (PRIVILEGED)**

**What it does:** The approver queue of pending home-location change requests for the whole tenant.

**How it is called:** Plain GET. Requires the attendance.approve privilege.

**What happens when called:** Lists every pending request across the tenant, for whoever reviews them.

**POST**  
**T**    **/api/tenant/wfh/location-change-requests/action**

**BEARER JWT (PRIVILEGED)**

**What it does:** Approves or rejects a home-location change request.

**How it is called:** POST JSON body { requestId, action: 'approve'|'reject' }. Requires attendance.approve.

**What happens when called:** If approved, actually updates the employee's registered home location and emails them the decision.

**GET**    **/api/tenant/wfh/stats**

**BEARER JWT (PRIVILEGED)**

**What it does:** Dashboard-level WFH statistics for the tenant.

**How it is called:** Plain GET. Requires reports.view.

**What happens when called:** Returns today's/this-month's WFH counts, pending approvals, a 30-day office-vs-WFH trend, and a role-wise breakdown.

**GET**    **/api/tenant/wfh/ledger**

**BEARER JWT (PRIVILEGED)**

**What it does:** A detailed, filterable log of every WFH check-in across the tenant.

**How it is called:** Plain GET.

**What happens when called:** Returns the raw WFH attendance rows for review/audit purposes.

**POST**  
**T**    **/api/tenant/users/:id/wfh-access**

**BEARER JWT (PRIVILEGED)**

**What it does:** Grants or revokes one specific employee's access to the WFH feature, independent of their role's default.

**How it is called:** POST to a specific user's ID.

**What happens when called:** Updates that user's individual privilege override.

## Breaks

Break session start/end and the daily time budget that governs them — the same budget the Earnings page now charges against if exceeded.

**GET**     **/api/breaks/active**

**BEARER JWT**

**What it does:** The caller's currently active break session, if any.

**How it is called:** Plain GET.

**What happens when called:** Simple lookup.

**GET**     **/api/breaks/today**

**BEARER JWT**

**What it does:** The caller's break sessions so far today plus their remaining time budget.

**How it is called:** Plain GET.

**What happens when called:** Sums today's completed break durations against the tenant's configured daily budget.

**POST**  
**T**     **/api/breaks/start**

**BEARER JWT**

**What it does:** Starts a break.

**How it is called:** POST JSON body { breakType?, lat, lng }.

**What happens when called:** Creates an active break session. Blocked if one is already active.

**POST**  
**T**     **/api/breaks/end**

**BEARER JWT**

**What it does:** Ends the active break.

**How it is called:** POST JSON body { lat, lng, clientTimestamp? }.

**What happens when called:** Closes the session. Stays flagged active (not fully closed) if the caller is currently outside the office geofence — a background scanner separately auto-ends and flags any break that exceeds the daily budget as a violation.

## QR Attendance

An alternate, faster check-in path: a manager-run rotating QR code that employees scan, still gated by the same policy checks (geofence, Wi-Fi, face, device trust) the tenant has configured for it.

**POST**  
**T**     **/api/qr/session/start**

**BEARER JWT (PRIVILEGED)**

**What it does:** Starts a QR display session (e.g. on a lobby tablet).

**How it is called:** POST. Requires QR display privilege.

**What happens when called:** Begins issuing a rotating signed token on a timer (interval configurable per tenant) for employees to scan.

**POST**  
**T**

**/api/qr/session/stop**

**BEARER JWT (PRIVILEGED)**

**What it does:** Stops an active QR display session.

**How it is called:** POST.

**What happens when called:** Ends the rotation.

**GET**

**/api/qr/current**

**BEARER JWT (PRIVILEGED)**

**What it does:** The current live QR token/image for display.

**How it is called:** Plain GET.

**What happens when called:** Returns whatever token is currently valid for the active session.

**POST**  
**T**

**/api/qr/validate**

**BEARER JWT**

**What it does:** An employee scans a QR code — validates it before proceeding to whatever additional checks the policy requires.

**How it is called:** POST JSON body with the scanned token.

**What happens when called:** Checks the token is current (not expired/rotated out) and returns which additional checks (geofence, face, Wi-Fi, device trust) this tenant's QR policy requires next.

**POST**  
**T**

**/api/attendance/mark-from-qr**

**BEARER JWT**

**What it does:** Records attendance after a successful QR scan and any required additional checks.

**How it is called:** POST with the scan context plus whatever extra verification the policy demanded.

**What happens when called:** Writes the attendance record, same authoritative path as the main check-in, tagged with attendanceMode 'qr'.

**POST**  
**T**

**/api/qr/scans/:id/override**

**BEARER JWT (PRIVILEGED)**

**What it does:** Manually approves a QR scan that failed automated verification.

**How it is called:** POST JSON body { reason } (required, logged).

**What happens when called:** Marks the scan as manually approved and logs the override with the given reason to the audit ledger.

**GET**

**/api/qr/history**

**BEARER JWT (PRIVILEGED)**

**What it does:** Session history for QR displays.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**GET**    **/api/qr/logs**

**BEARER JWT (PRIVILEGED)**

**What it does:** Raw scan logs, including failed attempts.

**How it is called:** Plain GET.

**What happens when called:** Read-only, for audit review.

**GET**    **/api/qr/config**

**BEARER JWT (PRIVILEGED)**

**What it does:** The tenant's current QR attendance policy.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**PUT**    **/api/qr/config**

**BEARER JWT (PRIVILEGED)**

**What it does:** Updates the tenant's QR attendance policy (rotation seconds, whether GPS/Wi-Fi/face/device-trust are required).

**How it is called:** PUT JSON body with the policy fields.

**What happens when called:** Saves the new policy; takes effect on the next scan.

## Leave Management

Leave requests, tenant leave policies, the approval queue, and manual balance adjustments.

**GET**    **/api/leave/mine**

**BEARER JWT**

**What it does:** The caller's own leave balance and request history.

**How it is called:** Plain GET.

**What happens when called:** Computes remaining balance per leave type and returns past requests.

**GET**    **/api/tenant/employees/:id/leave-balance**

**BEARER JWT (PRIVILEGED)**

**What it does:** The same balance view, for a specific employee, for admin lookup.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**

**/api/leave/requests**

**BEARER JWT**

**What it does:** Submits a new leave request.

**How it is called:** POST JSON body { policyId?, leaveType, startDate, endDate, reason, medicalCause?, halfDay? }.

**What happens when called:** Validates the date range and policy rules (advance notice, half-day allowed, no overlap with an existing pending/approved request), creates the request (auto-approved if the policy does not require approval), fires a webhook event, and emails whoever holds the approval privilege.

**GET**

**/api/tenant/leave/policies**

**BEARER JWT (PRIVILEGED)**

**What it does:** The tenant's configured leave policy types.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**

**/api/tenant/leave/policies**

**BEARER JWT (PRIVILEGED)**

**What it does:** Creates or updates a leave policy.

**How it is called:** POST with the policy fields.

**What happens when called:** Saves the policy.

**POST**  
**T**

**/api/tenant/leave/policies/seed-defaults**

**BEARER JWT (PRIVILEGED)**

**What it does:** One-click populates a starter set of standard leave policies for a brand-new tenant.

**How it is called:** POST, no body.

**What happens when called:** Inserts a predefined set of common leave types (sick, casual, etc.) if none exist yet.

**GET**

**/api/tenant/leave/requests**

**BEARER JWT (PRIVILEGED)**

**What it does:** The approver queue of leave requests for the tenant.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**

**/api/tenant/leave/requests/action**

**BEARER JWT (PRIVILEGED)**

**What it does:** Approves or rejects a leave request.

**How it is called:** POST JSON body { requestId, action, comment? }.

**What happens when called:** Updates the request's status, emails the employee the decision, and fires a webhook event.

**GET**    **/api/tenant/leave/adjustments**

**BEARER JWT (PRIVILEGED)**

**What it does:** History of manual leave-balance adjustments across the tenant.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**    **/api/tenant/leave/adjustments**

**BEARER JWT (PRIVILEGED)**

**What it does:** Manually adjusts an employee's leave balance (e.g. a carry-forward or a correction).

**How it is called:** POST with the employee, leave type, and delta.

**What happens when called:** Records the adjustment and updates their effective balance.

## Payroll & Compensation

Salary structure, generated payslip snapshots, tenant-wide payroll settings, and role-level default compensation templates.

**GET**    **/api/payroll/mine**

**BEARER JWT**

**What it does:** The caller's own current monthly payroll summary.

**How it is called:** GET, optional ?year=&month;=.

**What happens when called:** Computes gross/deductions/net for that period from their compensation profile plus approved leave in that month, live — not a stored snapshot.

**GET**    **/api/payroll/history**

**BEARER JWT**

**What it does:** The caller's permanent payslip history.

**How it is called:** Plain GET.

**What happens when called:** The first call for a given (employee, year, month) takes a permanent snapshot of that period's numbers; later salary changes never retroactively rewrite it. Returns every snapshot taken so far.

**GET**    **/api/payroll/history/:runId/pdf**

**BEARER JWT**

**What it does:** Downloads one specific payslip snapshot as a PDF.

**How it is called:** Plain GET with the run's ID.

**What happens when called:** Renders and streams back a formatted PDF payslip.

**GET**    **/api/tenant/payroll/settings**

**BEARER JWT (PRIVILEGED)**

**What it does:** The tenant's payroll-wide settings (working days/month, overtime rate, leave-penalty rules, etc.).

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**    **/api/tenant/payroll/settings**

**BEARER JWT (PRIVILEGED)**

**What it does:** Updates those tenant-wide payroll settings.

**How it is called:** POST with the changed fields.

**What happens when called:** Saves them; used in every payroll computation going forward.

**POST**  
**T**    **/api/tenant/payroll/employee/:userId**

**BEARER JWT (PRIVILEGED)**

**What it does:** Sets or updates one employee's individual compensation (CTC + salary components).

**How it is called:** POST JSON body { annualCtc, overtimeHourlyRate?, effectiveFrom?, components: [...] }.

**What happens when called:** Overwrites the active profile and its components with the new values, but first diffs the old vs. new state (CTC, and every component added/removed/changed) and permanently logs that diff to the compensation-history table — this is what the History pages below read. Notifies the employee their pay changed.

**GET**    **/api/tenant/payroll/employee/:userId/history**

**BEARER JWT (PRIVILEGED)**

**What it does:** The full recorded compensation change history for one employee, admin view.

**How it is called:** Plain GET.

**What happens when called:** Returns every logged change, newest first, with who changed what and when. Empty array (not an error) if nothing has ever changed.

**GET**    **/api/payroll/compensation-history/mine**

**BEARER JWT**

**What it does:** The same history, self-service — any employee can see their own.

**How it is called:** Plain GET.

**What happens when called:** Identical response shape to the admin endpoint above, scoped to the caller's own record.

**GET**    **/api/tenant/payroll/employee/:userId**

**BEARER JWT (PRIVILEGED)**

**What it does:** One employee's current compensation profile, admin view.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**GET**     **/api/tenant/payroll/overview**

**BEARER JWT (PRIVILEGED)**

**What it does:** A tenant-wide payroll summary across every employee.

**How it is called:** Plain GET.

**What happens when called:** Aggregates every employee's current numbers for the Payroll dashboard.

**GET**     **/api/tenant/payroll/role-defaults**

**BEARER JWT (PRIVILEGED)**

**What it does:** Role-level default compensation templates — set once per role, inherited by every employee in that role without an individual override.

**How it is called:** Plain GET.

**What happens when called:** Returns every configured role default plus which roles exist in the tenant.

**POST**  
**T**     **/api/tenant/payroll/role-defaults/:roleName**

**BEARER JWT (PRIVILEGED)**

**What it does:** Sets or updates a role's default compensation template.

**How it is called:** POST with CTC + components.

**What happens when called:** Saves the template; every employee in that role with no individual override immediately inherits it.

**DELETE**     **/api/tenant/payroll/role-defaults/:roleName**

**BEARER JWT (PRIVILEGED)**

**What it does:** Removes a role's default template.

**How it is called:** DELETE.

**What happens when called:** Employees in that role revert to having no compensation configured until one is set again.

## Earnings (Self-Service)

A day-by-day and monthly pay breakdown for the logged-in employee — hours worked, overtime, break-budget deductions, and leave-adjusted pay, computed live from real attendance data.

**GET**    **/api/earnings/mine**

**BEARER JWT**

**What it does:** The caller's own detailed earnings breakdown for one month.

**How it is called:** GET, optional ?year=&month;=.

**What happens when called:** For every day in the month, resolves status (present/absent/leave/holiday/weekend) from real attendance/break/leave data, computes hours worked, overtime beyond the tenant's shift length, minutes over the daily break budget and the resulting deduction, and leave pay impact (paid vs. chargeable, in date order against the tenant's monthly free-leave allowance). Rolls all of it into a monthly summary that reconciles exactly with /api/payroll/mine.

## Teams

A personal "my team" workspace for managers/leads (holders of the team.manage privilege), separate from tenant-wide administration.

**GET**    **/api/tenant/teams/mine**

**BEARER JWT (PRIVILEGED)**

**What it does:** The caller's own team and its members.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**    **/api/tenant/teams**

**BEARER JWT (PRIVILEGED)**

**What it does:** Creates the caller's team (one per manager).

**How it is called:** POST with a team name.

**What happens when called:** Creates the team row.

**GET**    **/api/tenant/teams/candidates**

**BEARER JWT (PRIVILEGED)**

**What it does:** Employees eligible to be added to the caller's team.

**How it is called:** Plain GET.

**What happens when called:** Filters to employees in the caller's own department not already on a team.

**POST**  
**T**    **/api/tenant/teams/members**

**BEARER JWT (PRIVILEGED)**

**What it does:** Adds a member to the caller's team.

**How it is called:** POST with the employee's ID.

**What happens when called:** Creates a team membership row.

**DELETE**

**/api/tenant/teams/members/:userId**

**BEARER JWT (PRIVILEGED)**

**What it does:** Removes a member from the caller's team.

**How it is called:** DELETE.

**What happens when called:** Deletes the membership row.

## Employee Directory & Departments

Tenant-wide staff listing, editing, and department management.

**GET**

**/api/tenant/employees**

**BEARER JWT (PRIVILEGED)**

**What it does:** The full employee directory for the tenant.

**How it is called:** Plain GET.

**What happens when called:** Returns every employee's profile, filtered to the caller's branch scope if they are branch-restricted.

**GET**

**/api/tenant/employees/:id**

**BEARER JWT (PRIVILEGED)**

**What it does:** One employee's full profile.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**PUT**

**/api/tenant/employees/:id**

**BEARER JWT (PRIVILEGED)**

**What it does:** Updates an employee's profile fields (department, designation, manager, status, etc.).

**How it is called:** PUT with the changed fields.

**What happens when called:** Saves the changes.

**GET**

**/api/tenant/departments**

**BEARER JWT (PRIVILEGED)**

**What it does:** The tenant's department list.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**

**/api/tenant/departments**

**BEARER JWT (PRIVILEGED)**

**What it does:** Creates a new department.

**How it is called:** POST with a name.

**What happens when called:** Creates the department row.

**GET**

**/api/employees/my-team**

**BEARER JWT**

**What it does:** The caller's own direct reports, if they are set as anyone's manager.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

## Branches & Shifts

Multi-location support: office geofence/network settings per branch, and shift schedules (including one-off overrides for specific employees/dates).

**GET**

**/api/branches**

**BEARER JWT (PRIVILEGED)**

**What it does:** Every branch in the tenant.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**GET**

**/api/branches/:id**

**BEARER JWT (PRIVILEGED)**

**What it does:** One branch's full configuration.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**

**/api/branches/bulk**

**BEARER JWT (PRIVILEGED)**

**What it does:** Creates multiple branches in one call (e.g. initial setup wizard).

**How it is called:** POST with an array of branches.

**What happens when called:** Inserts them all.

**POST**

**/api/branches**

BEARER JWT (PRIVILEGED)

**What it does:** Creates a single branch.

**How it is called:** POST with the branch fields.

**What happens when called:** Creates the branch and a default shift for it.

**PATCH**

**/api/branches/:id**

BEARER JWT (PRIVILEGED)

**What it does:** Updates a branch's configuration (geofence, Wi-Fi, etc.).

**How it is called:** PATCH with the changed fields.

**What happens when called:** Saves the changes.

**POST**

**/api/geocode/forward**

BEARER JWT

**What it does:** Converts a typed address into latitude/longitude, for setting up a branch or home location.

**How it is called:** POST with an address string.

**What happens when called:** Calls an external geocoding provider and returns coordinates.

**GET**

**/api/branches/:branchId/shifts**

BEARER JWT (PRIVILEGED)

**What it does:** Shifts defined for a branch.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**

**/api/branches/:branchId/shifts**

BEARER JWT (PRIVILEGED)

**What it does:** Creates a new shift for a branch.

**How it is called:** POST with shift times.

**What happens when called:** Creates the shift row.

**PATCH**

**/api/shifts/:id**

BEARER JWT (PRIVILEGED)

**What it does:** Updates a shift's times.

**How it is called:** PATCH.

**What happens when called:** Saves the changes.

**POST**

**/api/tenant/employees/:id/shift-override**

**BEARER JWT (PRIVILEGED)**

**What it does:** Sets a one-off different shift for one employee on specific date(s), without changing their default shift.

**How it is called:** POST with the override dates/shift.

**What happens when called:** Creates override row(s) that take precedence over the employee's normal shift on those dates.

**GET**

**/api/tenant/employees/:id/shift-overrides**

**BEARER JWT (PRIVILEGED)**

**What it does:** An employee's active shift overrides.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**DELETE**

**/api/tenant/employees/:id/shift-overrides/:overrideId**

**BEARER JWT (PRIVILEGED)**

**What it does:** Removes a shift override.

**How it is called:** DELETE.

**What happens when called:** Employee reverts to their default shift on that date.

## Roles & Permissions (RBAC)

The privilege system: what each role can do by default, per-employee overrides, and the feature catalog those privileges are drawn from.

**GET**

**/api/tenant/feature-catalog**

**BEARER JWT**

**What it does:** The full list of grantable privilege strings and what each one unlocks.

**How it is called:** Plain GET.

**What happens when called:** Read-only, static reference used to build the Roles & Permissions UI.

**GET**

**/api/tenant/my-privileges**

**BEARER JWT**

**What it does:** The caller's own effective privilege list.

**How it is called:** Plain GET.

**What happens when called:** Used by the frontend to decide which sidebar items/actions to show, though every privileged endpoint above independently re-checks this server-side regardless.

**GET**    **/api/tenant/my-branches**

**BEARER JWT**

**What it does:** Which branches the caller is scoped to (or unrestricted, for admins).

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**GET**    **/api/tenant/roles**

**BEARER JWT (PRIVILEGED)**

**What it does:** Every role that exists in the tenant plus its default privilege set.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**    **/api/tenant/roles**

**BEARER JWT (PRIVILEGED)**

**What it does:** Creates a new custom role name with a default privilege set.

**How it is called:** POST with the role name and privileges.

**What happens when called:** Creates the role-default row.

**PATCH**    **/api/tenant/roles/:id**

**BEARER JWT (PRIVILEGED)**

**What it does:** Updates a role's default privilege set.

**How it is called:** PATCH with the new privilege list.

**What happens when called:** Saves it; takes effect immediately for everyone currently in that role, unless they have an individual override.

## Tenant Administration

Company-wide policy, staff onboarding, notifications, and device-change approvals for the caller's own tenant.

**GET**    **/api/tenant/analytics**

**BEARER JWT (PRIVILEGED)**

**What it does:** Today's/this-month's attendance snapshot for the tenant.

**How it is called:** Plain GET.

**What happens when called:** Read-only aggregate.

**GET**    **/api/tenant/analytics/trends**

**BEARER JWT (PRIVILEGED)**

**What it does:** Historical attendance trend data.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**    **/api/tenant/users/create**

**BEARER JWT (PRIVILEGED)**

**What it does:** Hires a new employee or manager.

**How it is called:** POST JSON body { email, name, role, privileges? }.

**What happens when called:** Creates the account with a temporary password, emails the credentials, and auto-registers the role name in the tenant's role list if it is a brand-new custom role.

**GET**    **/api/tenant/users**

**BEARER JWT (PRIVILEGED)**

**What it does:** List staff in the tenant.

**How it is called:** Plain GET.

**What happens when called:** Read-only, used by several admin screens.

**POST**  
**T**    **/api/tenant/users/:id/qr-access**

**BEARER JWT (PRIVILEGED)**

**What it does:** Grants or revokes one employee's QR-attendance-specific access, independent of their role default.

**How it is called:** POST.

**What happens when called:** Updates their individual privilege override.

**GET**    **/api/tenant/config**

**BEARER JWT (PRIVILEGED)**

**What it does:** The tenant's full policy configuration (geofence, Wi-Fi, shift/break rules, WFH policy).

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**    **/api/tenant/config/update**

**BEARER JWT (PRIVILEGED)**

**What it does:** Updates any subset of that tenant policy.

**How it is called:** POST with the changed fields.

**What happens when called:** Saves them; every future attendance check uses the new values immediately.

**GET**    **/api/tenant/policy**

**BEARER JWT**

**What it does:** The tenant-wide policy announcement text shown to all employees.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**PUT**    **/api/tenant/policy**

**BEARER JWT (PRIVILEGED)**

**What it does:** Sets the policy announcement text.

**How it is called:** PUT with the text.

**What happens when called:** Saves it; visible to every employee immediately.

**GET**    **/api/tenant/notifications**

**BEARER JWT**

**What it does:** The caller's own notification feed.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**    **/api/tenant/notifications/:id/read**

**BEARER JWT**

**What it does:** Marks one notification as read.

**How it is called:** POST.

**What happens when called:** Updates that notification's read flag.

**POST**  
**T**    **/api/tenant/notifications/read-all**

**BEARER JWT**

**What it does:** Marks every notification as read.

**How it is called:** POST.

**What happens when called:** Bulk update.

**GET**    **/api/tenant/device-requests**

**BEARER JWT (PRIVILEGED)**

**What it does:** Pending device-change requests (an employee asking to switch their pinned device).

**How it is called:** Plain GET.

**What happens when called:** Read-only.

POST

/api/tenant/device-requests/action

BEARER JWT (PRIVILEGED)

**What it does:** Approves or rejects a device-change request.

**How it is called:** POST { requestId, action }.

**What happens when called:** If approved, re-pins the employee's account to their new device ID.

## Holidays

The company holiday calendar, plus employee-chosen optional/floating holidays within a tenant-configured limit.

GET

/api/tenant/holidays

BEARER JWT

**What it does:** The holiday calendar.

**How it is called:** Plain GET.

**What happens when called:** Read-only, feeds into attendance status (a check-in-less holiday shows as "Holiday" not "Absent").

POST

/api/tenant/holidays

BEARER JWT (PRIVILEGED)

**What it does:** Adds a holiday.

**How it is called:** POST { date, name }.

**What happens when called:** Creates the holiday row.

DELETE

/api/tenant/holidays/:id

BEARER JWT (PRIVILEGED)

**What it does:** Removes a holiday.

**How it is called:** DELETE.

**What happens when called:** Deletes the row.

GET

/api/tenant/holidays/optional

BEARER JWT

**What it does:** The caller's chosen optional/floating holidays this year.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**

**/api/tenant/holidays/optional**

**BEARER JWT**

**What it does:** Chooses an optional holiday, within the tenant's configured yearly limit.

**How it is called:** POST with the date.

**What happens when called:** Records the choice if the limit has not been used up yet.

**POST**  
**T**

**/api/tenant/holidays/import-public**

**BEARER JWT (PRIVILEGED)**

**What it does:** Bulk-imports a country/region's public holidays for the year.

**How it is called:** POST with a country/region code.

**What happens when called:** Fetches and inserts the standard public holiday list.

## Alerts, Corrections & Approvals

The unified review queues a manager/admin works through: timing/fraud alerts, attendance corrections, and pending (late/WFH) check-ins awaiting sign-off.

**GET**

**/api/tenant/alerts**

**BEARER JWT (PRIVILEGED)**

**What it does:** Timing/fraud alerts (break overstay, geofence exit, etc.) awaiting review.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**

**/api/tenant/alerts/action**

**BEARER JWT (PRIVILEGED)**

**What it does:** Accepts or rejects an alert.

**How it is called:** POST { alertId, action }.

**What happens when called:** Resolves the alert.

**GET**

**/api/attendance/corrections/mine**

**BEARER JWT**

**What it does:** (listed under Attendance above) the caller's own correction request history.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**GET**    **/api/tenant/corrections**

**BEARER JWT (PRIVILEGED)**

**What it does:** The approver queue of attendance correction requests.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**    **/api/tenant/corrections/action**

**BEARER JWT (PRIVILEGED)**

**What it does:** Approves or rejects a correction request.

**How it is called:** POST { correctionId, action }.

**What happens when called:** Resolves the request; does not silently rewrite the original attendance row, preserving the audit trail.

**GET**    **/api/tenant/attendance/pending**

**BEARER JWT (PRIVILEGED)**

**What it does:** The approver queue for pending check-ins — both late-arrival and WFH check-ins land here.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**    **/api/tenant/attendance/action**

**BEARER JWT (PRIVILEGED)**

**What it does:** Approves or rejects a pending check-in.

**How it is called:** POST { logId, action }.

**What happens when called:** Finalizes that attendance row's status.

## Audit Ledger

An immutable, hash-chained log of privileged actions across the tenant — every entry cryptographically links to the previous one, so tampering with history is detectable.

**GET**    **/api/tenant/ledger**

**BEARER JWT (PRIVILEGED)**

**What it does:** The full audit ledger for the tenant.

**How it is called:** Plain GET. Requires reports.view.

**What happens when called:** Read-only.

**POST**  
**T**

**/api/tenant/ledger/verify**

**BEARER JWT (PRIVILEGED)**

**What it does:** Verifies the hash chain has not been tampered with.

**How it is called:** POST, no body.

**What happens when called:** Recomputes every entry's hash against the previous one and reports whether the chain is intact, and which block(s) failed if not.

## Super Admin

Platform-wide oversight, restricted to the role: super\_admin only — approving new tenants, suspending/reinstating them, and cross-tenant analytics.

**GET**

**/api/super/requests**

**BEARER JWT (PRIVILEGED)**

**What it does:** Every pending tenancy sign-up request.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**GET**

**/api/super/notifications**

**BEARER JWT (PRIVILEGED)**

**What it does:** Platform-wide notifications for the super admin.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

**POST**  
**T**

**/api/super/approve**

**BEARER JWT (PRIVILEGED)**

**What it does:** Approves a tenancy request.

**How it is called:** POST { requestId, plan?, featuresAllowed? }.

**What happens when called:** Provisions a brand-new tenant plus its first tenant\_admin account, and emails that admin their login credentials.

**GET**

**/api/super/tenants**

**BEARER JWT (PRIVILEGED)**

**What it does:** Every tenant on the platform with live employee counts.

**How it is called:** Plain GET.

**What happens when called:** Read-only.

POST  
T

/api/super/tenants/status

BEARER JWT (PRIVILEGED)

**What it does:** Suspends or reactivates a tenant.

**How it is called:** POST { tenantId, status }.

**What happens when called:** Suspended tenants can no longer log in or check in until reactivated.

POST  
T

/api/super/tenants/delete

BEARER JWT (PRIVILEGED)

**What it does:** Permanently deletes a tenant and its data.

**How it is called:** POST { tenantId }.

**What happens when called:** Irreversible — removes the tenant and everything scoped to it.

GET

/api/super/analytics

BEARER JWT (PRIVILEGED)

**What it does:** Organization-wide analytics across every tenant.

**How it is called:** Plain GET.

**What happens when called:** Read-only aggregate.

## Integrations: Service Accounts (API Keys)

Machine-to-machine credentials for partner integrations, scoped to an explicit privilege list — an alternative to the human-login JWT above. tenant\_admin only.

GET

/api/tenant/service-accounts

BEARER JWT (PRIVILEGED)

**What it does:** Lists the tenant's service accounts (metadata only — the key itself is never retrievable again after creation).

**How it is called:** Plain GET.

**What happens when called:** Read-only.

POST  
T

/api/tenant/service-accounts

BEARER JWT (PRIVILEGED)

**What it does:** Creates a new service account / API key.

**How it is called:** POST { name, privileges: [...] }.

**What happens when called:** Generates a long-lived key (prefix stk\_live\_), stores only its bcrypt hash, and returns the raw key exactly once. The requested privileges must be a subset of what the creator themselves holds. Once created, calling apps authenticate by sending Authorization: Bearer <stk\_live\_...> on any endpoint instead of a JWT.

**DEL  
ETE**

**/api/tenant/service-accounts/:id**

**BEARER JWT (PRIVILEGED)**

**What it does:** Revokes a service account key immediately.

**How it is called:** DELETE.

**What happens when called:** The key stops working on the very next request; irreversible (a new key must be issued to replace it).

## Integrations: Webhooks

Event subscriptions so a partner app can react to real events instead of polling — every delivery is HMAC-signed so the receiver can verify it genuinely came from this server.

**GET**

**/api/tenant/webhooks**

**BEARER JWT (PRIVILEGED)**

**What it does:** Lists the tenant's webhook subscriptions (the signing secret is never returned again after creation).

**How it is called:** Plain GET.

**What happens when called:** Also returns the full list of subscribable event names.

**POST**

**/api/tenant/webhooks**

**BEARER JWT (PRIVILEGED)**

**What it does:** Creates a new webhook subscription.

**How it is called:** POST { url, events: [...] } — events are drawn from: attendance.checked\_in, attendance.checked\_out, leave.requested, leave.approved, leave.rejected.

**What happens when called:** Generates a signing secret, returned exactly once. From then on, whenever a subscribed event happens elsewhere in the API, this URL receives a POST with body {event, data, timestamp} and header X-SmartTeams-Signature: hex(HMAC-SHA256(signingSecret, rawBody)) — delivery is fire-and-forget with a 5-second timeout and no retry queue yet.

**DEL  
ETE**

**/api/tenant/webhooks/:id**

**BEARER JWT (PRIVILEGED)**

**What it does:** Deletes a webhook subscription.

**How it is called:** DELETE.

**What happens when called:** Stops all future deliveries to that URL immediately.